This is our groups project topic in algorithm engineering sessional.

This things should be cover before next checkpoint:

1. Define your problem clearly.
2. Show polynomial-time reductions to and from other known hard problems.
3. Create a table of existing algorithms for the problem, including:
   Exact exponential algorithms
   Approximation algorithms
   Randomized algorithms
   Heuristic algorithms
   Metaheuristic algorithms
4. Identify at least two algorithms from this list that you will implement.
5. Design experiments to evaluate your implementations.
6. Discuss both theoretical and real-world applications of your problem.

**If you want, you can also show some potential research direction (this is optional) that we can further work with.

Please do deep research and help me complete our projects checkpoint.

I'll now generate a comprehensive report for your Weapon-Target Assignment Problem project checkpoint.

Weapon-Target Assignment Problem: Comprehensive Research Report for Algorithm Engineering Checkpoint

1. Problem Definition

The **Weapon-Target Assignment (WTA) problem** is a fundamental combinatorial optimization problem in military operations research that seeks to optimally assign a set of weapons to a set of targets to minimize the expected value of surviving targets (or equivalently, maximize expected damage).[1] [2]

1.1 Mathematical Formulation

Decision Variables:

- x_{ij} = number of weapons of type i assigned to target j

Parameters:

- $W = \{1, \ldots, m\}$: set of weapon types
- $T = \{1, \ldots, n\}$: set of targets
- v_j: value (importance) of target j
- p_{ij}: probability that one weapon of type i destroys target j
- μ_i: number of available weapons of type i

Objective Function (Static WTA):

$$\min \sum_{j \in T} v_j \prod_{i \in W} (1 - p_{ij})^{x_{ij}}$$

This minimizes the expected total value of targets that survive all weapon assignments, where $\prod_{i \in W}(1 - p_{ij})^{x_{ij}}$ represents the survival probability of target j.[2] [1]

Constraints:

$$\sum_{j \in T} x_{ij} \leq \mu_i, \quad \forall i \in W \quad \text{(weapon availability)}$$

$$x_{ij} \in \mathbb{Z}_+, \quad \forall i \in W, j \in T \quad \text{(integer assignment)}$$

1.2 Problem Variants

Static vs. Dynamic WTA:

- **Static WTA (SWTA)**: Single-stage assignment made once before engagement
- **Dynamic WTA (DWTA)**: Multi-stage assignment with observation of outcomes between stages ("shoot-look-shoot")[3] [4]

Objective Variants:

- **Asset-based (defensive)**: Maximize survival of defended assets under attack[5]
- **Target-based (offensive)**: Minimize expected value of surviving hostile targets[2]
- **Multi-objective**: Simultaneously optimize damage, cost, resource consumption[6] [7]

Extensions:

- **Sensor-Weapon-Target Assignment (SWTA)**: Adds sensor allocation layer[4] [8]
- **Constrained WTA**: Engagement time windows, setup times, platform limitations[9] [10]

2. Polynomial-Time Reductions and Complexity

2.1 NP-Completeness

The WTA problem is **NP-complete**, even in relatively restricted forms. This means:[11] [12] [13] [1]

1. No polynomial-time exact algorithm exists (unless P=NP)
2. Any instance can be verified in polynomial time
3. The problem is at least as hard as all other NP problems

2.2 Reduction from SUBSET PRODUCT Problem

Theorem: The WTA problem is NP-hard.[14]

Proof Sketch (Reduction from SUBSET PRODUCT):

Given a set of positive integers $A = \{a_1, a_2, \ldots, a_m\}$ and target product b, construct a WTA instance with:

- **2 targets** with equal values $V_1 = V_2 = P = \prod_{i=1}^{m} a_i$
- $m + 2$ **weapons**: $W_1, \ldots, W_m, W_\alpha, W_\beta$
- **Survival probabilities**:
  - For weapons $W_1, \ldots, W_m$: $q_{i,1} = q_{i,2} = \frac{1}{a_i}$
  - For weapon W_α: $q_{\alpha,1} = q_{\alpha,2} = \frac{b}{2P}$
  - For weapon W_β: $q_{\beta,1} = q_{\beta,2} = \frac{1}{2b}$
- **Threshold**: $k = 1$

Key insight: The WTA instance has a solution achieving expected surviving value ≤ 1 if and only if there exists a subset $X \subseteq \{1, \ldots, m\}$ such that $\prod_{x \in X} a_x = b$.

The proof establishes that solving WTA in polynomial time would solve SUBSET PRODUCT in polynomial time, contradicting the assumption that SUBSET PRODUCT is NP-complete.[14]

2.3 Reductions To and From Other Problems

WTA reduces TO:

- **Mixed-Integer Linear Programming (MILP)**: Via piecewise linear approximation or probability chains[15] [16]
- **Network flow problems**: For special cases (uniform probabilities)[2]
- **Nonlinear integer programming**: Direct formulation[1]

WTA reduces FROM:

- **Knapsack problem**: Similar resource allocation structure[1]
- **Set cover**: Combinatorial assignment patterns[1]

- **Generalized assignment problem**: Task-agent matching[17]

Related NP-complete problems:

- **Target Assignment and Path Finding (TAPF)**: Adds path-planning constraints[18]
- **Multi-Agent Path Finding (MAPF)**: Special case when targets are pre-assigned[19]

3. Table of Existing Algorithms

Algorithm Category	Algorithm Name	Complexity / Features	Solution Quality	References
Exact Exponential	Exhaustive Search	$O(\|T\|^{\|W\|})$ - exponential	Optimal	[5] [20]
	Branch-and-Bound	Worst-case exponential; up to 80×80 instances	Optimal with bounds	[21] [2] [22]
	Branch-and-Adjust	Specialized B&B with convex under-approximation	Optimal; handles 1500×1000	[1] [23]
	Branch-Price-and-Cut	Column generation; solves 10000+ instances	Optimal; seconds for large instances	[24] [25]
	MILP Solvers (Gurobi/CPLEX)	Probability chains or linearization	Optimal/near-optimal	[15] [16]
Approximation	LP Relaxation + Rounding	$O(\text{poly}(m, n))$	Suboptimal; feasibility issues	[21] [2]
	Piecewise Linear Approx.	MILP with breakpoints	Provable bounds (δ-error)	[1] [15]
	Network Flow Heuristics	Special cases (uniform p_{ij})	Optimal for restricted variants	[2]
	Greedy (1-1/e)-Approx.	Submodular objectives only	(1-1/e)-approximation guarantee	[26]
Randomized	Monte Carlo Sampling	Statistical sampling	Baseline quality	[27] [28]
	Randomized Constructive	Multiple runs with statistics	Variable quality	[29]
Heuristic	Maximum Marginal Return (MMR)	$O(\|W\| \times \|T\|)$	Fast; optimal for uniform p_{ij}	[5] [2]
	Enhanced MMR (EMMR)	$O(\|W\|^2 \times \|T\|)$	Better than MMR	[5]
	Very Large-Scale Neighborhood Search	Multi-exchange via improvement graphs	High quality; up to 200×200	[2] [30]

Algorithm Category	Algorithm Name	Complexity / Features	Solution Quality	References
	Rule-Based / Fuzzy Logic	Domain-specific rules	Interpretable; fast	[31] [32]
Metaheuristic	Genetic Algorithm (GA)	Population-based; crossover + mutation	Good for medium-large; often seeded	[33] [34] [35]
	Simulated Annealing (SA)	Temperature-based; probabilistic acceptance	Good exploration	[33] [36] [37]
	Parallel SA (GPU)	Multi-start on CUDA	250× speedup over CPU	[37]
	Tabu Search	Memory-based local search	Effective for DWTA	[38] [39]
	Particle Swarm Optimization (PSO)	Swarm intelligence	Often outperforms GA	[12] [40]
	Ant Colony Optimization (ACO)	Pheromone-based	Competitive with NSGA-II	[12] [41] [42]
	Variable Neighborhood Search (VNS)	Systematic neighborhood change	Good balance	[33] [22]
	Crow Search Algorithm	Modified swarm approach	Beats several baselines	[43]
	Hybrid (PSO+TS, GA+SA)	Combined metaheuristics	Improved convergence	[38] [34] [44]
	Cross-Entropy Method	Estimation → convex optimization	Better than heuristic/MC	[27] [28]
	Adaptive Large Neighborhood Search	Model-driven repair	Tight bounds; competitive	[45]
Multi-Objective	MOEA/D	Decomposition-based	Pareto front generation	[7] [46]
	NSGA-II, NSGA-III	Non-dominated sorting	Well-established Pareto sets	[6] [47]
	D-NSGA-III-A	Dominance degree matrix	Faster non-dominated sorting	[6]
	MPACO (Modified Pareto ACO)	Multi-objective ACO	Beats NSGA-II, SPEA-II	[41] [42]
	MOABC (Multi-Obj. Artificial Bee Colony)	Bee colony for multi-objective	Good convergence/distribution	[48] [49]
Learning-Based / RL	Deep Q-Learning (DQN)	Neural Q-function approximation	Handles large-scale; avoids local optima	[50]

Algorithm Category	Algorithm Name	Complexity / Features	Solution Quality	References
	DDPG-DNPE	Deep deterministic policy gradient	Better than DQN for dynamic WTA	[51]
	Transformer Networks	Attention mechanisms	High quality; scalable for dynamic	[52]
	Multi-Agent RL (MADDPG)	Decentralized cooperative	UAV swarm coordination	[53]
	TANet-TD3	Target assignment network + TD3	Simultaneous assignment & path planning	[53]
	LLM-Driven WTA	Large language model reasoning	Contextual, human-like prioritization	[54] [55]
	Graph Neural Networks	Relational structure modeling	Scalable, data-efficient	[56]
	Hopfield Neural Network	Energy function minimization	Fast convergence; Pareto front	[57]

4. Algorithm Selection for Implementation

For your project, we recommend implementing **at least two algorithms** from different categories to demonstrate diverse solution approaches:

4.1 Recommended Algorithm Pair

Algorithm A: Maximum Marginal Return (MMR) Heuristic[5] [2]

- **Type**: Deterministic greedy heuristic
- **Complexity**: $O(|W| \times |T|)$
- **Implementation**: Straightforward; ~50 lines of code
- **Advantages**: Very fast, optimal for special cases, good baseline
- **Pseudocode**:

```
def MMR(weapons, targets, kill_probs, values):
    assignment = {}
    for weapon in weapons:
        best_marginal_return = -1
        best_target = None
        for target in targets:
            marginal_return = values[target] * kill_probs[weapon][target]
            if marginal_return > best_marginal_return:
                best_marginal_return = marginal_return
                best_target = target
        assignment[weapon] = best_target
        values[best_target] *= (1 - kill_probs[weapon][best_target])
    return assignment
```

Algorithm B: Simulated Annealing (SA) Metaheuristic[33] [36]

- **Type**: Randomized metaheuristic
- **Implementation**: ~100 lines with temperature schedule
- **Advantages**: Escapes local optima, quality-time tradeoff, easy parallelization
- **Pseudocode**:

```
def SimulatedAnnealing(initial_solution, T_init, alpha, max_iter):
    current = initial_solution
    best = current
    T = T_init
    for i in range(max_iter):
        neighbor = generate_neighbor(current)  # Swap two assignments
        delta = objective(neighbor) - objective(current)
        if delta < 0 or random() < exp(-delta / T):
            current = neighbor
            if objective(current) < objective(best):
                best = current
        T *= alpha  # Cool down
    return best
```

4.2 Optional Third Algorithm (for comparison)

Algorithm C: Genetic Algorithm (GA)[35] [33]

- Population-based evolutionary search
- Demonstrates population diversity vs. local search
- Complexity: $O(\text{generations} \times \text{pop_size} \times |W| \times |T|)$

Alternative C: Branch-and-Bound (via MILP solver)[16] [15]

- Use PuLP/Pyomo with GLPK or SCIP (open-source)
- Provides "near-optimal" benchmark for small instances
- Shows exact vs. heuristic quality gap

5. Experimental Design

5.1 Instance Generation

Parameters to vary:

1. **Problem size**: $(|W|, |T|) \in \{(20, 20), (40, 40), (80, 80), (160, 160)\}$[1]
2. **Weapon availability**: $\mu_i \in [1, 5]$ per weapon type
3. **Kill probabilities**: $p_{ij} \sim \text{Uniform}(0.3, 0.9)$[1]
4. **Target values**: $v_j \sim \text{Uniform}(1, 100)$ or priority-based[33]
5. **Density**: Percentage of engageable pairs (sparse vs. dense)[1]

Instance types:

- **Easy**: Plenty of weapons, high kill probabilities
- **Medium**: Balanced resources
- **Hard**: Scarce weapons, low kill probabilities, many high-value targets

5.2 Algorithms to Compare

At minimum:

1. **MMR** (greedy baseline)
2. **Simulated Annealing** (your primary metaheuristic)
3. Optionally: **GA** or **MILP** (quality benchmark)

5.3 Evaluation Metrics

Metric	Description	Formula/Method				
Objective Value	Expected surviving target value (lower is better)	$\sum_j v_j \prod_i (1-p_{ij})^{x_{ij}}$				
Computational Time	Wall-clock time to solution (seconds)	Average over 20 runs for randomized algorithms				
Optimality Gap	Distance from best-known solution	$\frac{\text{alg_obj} - \text{best_obj}}{\text{best_obj}} \times 100\%$				
Success Rate	% of runs finding a "good" solution	Count runs within 5% of best				
Scalability	Performance vs. problem size	Plot time/quality vs. ((	W	,	T	))

5.4 Experimental Procedure

1. **Generate instances**: Create 10 instances for each size/density combination
2. **Run each algorithm**:
   - MMR: 1 run (deterministic)
   - SA: 20 runs (report mean ± std)
   - GA: 20 runs
3. **Record**:
   - Best objective found
   - Average objective (for randomized)
   - Time per run
   - Number of iterations/generations
4. **Statistical analysis**:

    - Paired t-tests to compare algorithm means
    - Wilcoxon signed-rank for non-normal distributions
5. **Visualizations**:
    - Box plots: objective value distribution per algorithm
    - Line plots: time vs. problem size
    - Scatter: time vs. quality trade-off

5.5 Parameter Tuning

Simulated Annealing:

- Initial temperature T_0: Set so ~90% of worse moves accepted initially
- Cooling rate $\alpha \in [0.85, 0.99]$
- Max iterations: Proportional to $|W| \times |T|$

Genetic Algorithm:

- Population size: 50-100
- Crossover rate: 0.7-0.9
- Mutation rate: 0.01-0.1
- Selection: Tournament (size 3)

Use grid search or pilot experiments on 3-5 small instances to select parameters.

5.6 Implementation Tools

- **Language**: Python (recommended) or C++/Java
- **Libraries**:
    - NumPy/SciPy for computations
    - Matplotlib/Seaborn for visualization
    - PuLP/Pyomo for MILP (optional)
- **Hardware**: Record CPU specs (for reproducibility)
- **Random seeds**: Fix seeds for reproducibility

6. Theoretical and Real-World Applications

6.1 Theoretical Significance

Computational Theory:

- **Benchmark for NP-complete problems**: Tests new algorithm paradigms (e.g., LLMs for optimization)[54]
- **Nonlinear integer optimization**: Canonical case study for resource allocation

- **Submodularity analysis**: Objective is submodular under certain conditions, enabling approximation algorithms[26]

Operations Research:

- **Dynamic programming**: Multi-stage DWTA models sequential decision-making under uncertainty
- **Game theory**: Two-sided WTA represents adversarial resource allocation[58]
- **Multi-objective optimization**: Real combat scenarios require Pareto front analysis (damage vs. cost)[7] [6]

6.2 Real-World Military Applications

1. Air and Missile Defense[59] [60] [10] [5]

- **Ballistic missile defense**: Allocating interceptors to incoming ICBMs/SLBMs
- **Surface-to-air missiles (SAM)**: Engaging hostile aircraft, helicopters, cruise missiles
- **Anti-aircraft gun-missile formations**: Coordinated engagement[59]
- **Fleet air defense**: Multi-ship task groups defending against saturation attacks[61] [62]
- **Land-based air defense**: Task group protection in expeditionary operations[10]

2. Counter Rocket, Artillery, and Mortar (C-RAM)[63] [64] [5]

- **Israeli Iron Dome**:[65]
  - Intercepts short-range rockets (4-70 km)
  - Radar detects launch, calculates impact point
  - Only engages threats to designated areas (cost-effectiveness)
  - Successfully defends population centers (Sderot, Ashkelon) against Hamas/Hezbollah rockets
- **US Phalanx C-RAM**:[64]
  - 20mm HEIT-SD rounds (tungsten penetrators)
  - Deployed in Iraq/Afghanistan to protect bases from insurgent RAM
- **DRACO**: 76mm gun on wheeled/tracked platforms
- **Iron Beam**: Directed-energy laser C-RAM (7 km range)[64]

3. Naval Combat Management[62] [58]

- **Layered defense**: Long/medium-range SAM → point defense missiles → Close-In Weapons Systems (CIWS) → electronic warfare
- **Multi-ship coordination**: Cooperative engagement capability (CEC)[61]
- **Constraints**: Ship maneuverability, weapon blind sectors, radar horizon, setup times[62]

4. Drone Swarm Operations[66] [67]

- **Low-altitude threats**: Consumer/military drones, kamikaze UAVs

- **Heading error considerations**: Launcher orientation affects kill probability[66]
- **Cooperative UAV assignment**: Swarm-to-target allocation with path planning[53]

5. Ground Target Engagement[35]

- **Ground Target Attacking WTA (GTA-WTA)**: Aircraft/artillery assigning munitions to ground targets
- **Saturated attack**: More targets than available weapons → prioritization critical

6. Sensor-Weapon-Target Assignment (SWTA)[45] [8] [4]

- **Adds sensor layer**: Radar/EO sensors for detection and tracking
- **Objective**: Minimize total resource cost while achieving desired probability of target destruction[45]
- **Constraints**: Sensor coverage, communication bandwidth

6.3 Non-Military Applications

Search and Rescue[20]

- Assign heterogeneous search assets (dogs, aircraft, walkers) to partitioned search areas
- Minimize probability of not finding lost object/person

Cybersecurity

- Assign defensive resources (firewalls, intrusion detection systems) to network segments
- Minimize expected damage from cyber threats

Healthcare Resource Allocation

- Assign medical personnel/equipment to patients in triage scenarios
- Minimize expected casualties

7. Potential Research Directions (Optional)

7.1 Deep Reinforcement Learning & Neural Approaches[50] [51] [68] [55] [52]

Current State:

- DQN, DDPG handle large-scale dynamic WTA
- Transformer architectures for multi-stage scenarios[52]
- Multi-agent RL for cooperative swarm engagements[53]

Open Problems:

- **Generalized AI**: Large Language Models for tactical reasoning[55] [54]
  - LLM-driven WTA treats assignment as contextual reasoning task
  - Human-like prioritization without hand-tuned weights

- **Your contribution**: Benchmark LLMs vs. traditional optimization on standard WTA instances

- **Graph Neural Networks**: Exploit relational structure (weapon-target bipartite graphs)[56]
  - **Your contribution**: Design GNN architectures that learn assignment policies
- **Explainable RL**: Current deep RL lacks interpretability for military commanders
  - **Your contribution**: Hybrid models combining RL policy with rule extraction

7.2 Multi-Agent Path Finding Integration[19] [18] [53]

Current State:

- Target Assignment and Path Finding (TAPF) extends WTA with collision avoidance
- Conflict-Based Search with Target Assignment (CBS-TA) for optimal TAPF[19]

Open Problems:

- **Real-time TAPF**: CBS-TA scales poorly; need faster algorithms
  - **Your contribution**: Develop anytime CBS-TA variants with quality guarantees
- **Cooperative Multi-Agent Path Finding (CoMAPF)**: Agents coordinate on joint tasks[69]
  - **Your contribution**: Model WTA with cooperative engagements (two weapons must simultaneously engage)
- **Dynamic obstacles**: Current TAPF assumes static environment
  - **Your contribution**: Integrate predictive models of target evasion

7.3 Large-Scale Exact Algorithms[24] [25]

Current State:

- Branch-price-and-cut solves 10,000+ weapon/target instances in seconds[24]
- Column generation enables decomposition[70]

Open Problems:

- **Parallel exact algorithms**: Exploit multi-core/GPU for branch-and-bound
  - **Your contribution**: Implement distributed branch-and-adjust on GPU clusters
- **Warm-start techniques**: Reuse solutions from previous time steps in DWTA
  - **Your contribution**: Develop solution repair algorithms for dynamic scenarios

7.4 Uncertainty and Robustness[71] [60]

Current State:

- Fuzzy multi-objective programming for battlefield uncertainty[60]
- Interval numbers for uncertain kill probabilities[71]

Open Problems:

- **Robust optimization**: Worst-case guarantees against adversarial behavior
  - **Your contribution**: Formulate robust WTA with uncertainty sets on p_{ij}
- **Stochastic programming**: Two-stage WTA with random target arrivals
  - **Your contribution**: Sample average approximation (SAA) algorithms
- **Online learning**: Update kill probability estimates during battle
  - **Your contribution**: Bayesian updating framework integrated into DWTA

7.5 Multi-Objective and Many-Objective Optimization[72] [6] [7]

Current State:

- MOEA/D, NSGA-III generate Pareto fronts for 2-3 objectives
- Bi-objective multi-stage WTA with limited ammunition[72]

Open Problems:

- **Many-objective WTA (4+ objectives)**: Damage, cost, time, collateral, fuel, etc.
  - **Your contribution**: Decomposition-based MOEAs for 5+ objectives
- **Dynamic objective weights**: Commander preferences change during battle
  - **Your contribution**: Interactive MOEA with real-time re-weighting
- **Fairness constraints**: Ensure no asset is over-defended or under-defended
  - **Your contribution**: Fair multi-objective optimization formulations

7.6 Explainable AI and Human-in-the-Loop[32] [73]

Open Problems:

- **Rule extraction from neural networks**: Train NN on expert data, extract interpretable rules[73]
  - **Your contribution**: Decision tree approximation of trained RL policies
- **Confidence scores**: Provide uncertainty quantification for AI recommendations
  - **Your contribution**: Bayesian neural networks for WTA with credible intervals
- **Commander override**: Allow human to modify AI suggestions with minimal recomputation
  - **Your contribution**: Fast re-optimization algorithms for constraint addition

7.7 Benchmark Development

Open Problems:

- **Standardized datasets**: Public benchmark instances like TSPLIB for TSP
  - **Your contribution**: Curate realistic WTA instance library with ground truth optimal solutions[74]
- **Reproducibility**: Open-source implementations of 10+ WTA algorithms

- **Your contribution**: Python package "WTAlib" with unified interface

7.8 Theoretical Advances

Open Problems:

- **Approximation guarantees beyond submodular cases**: Current (1-1/e) only applies to submodular objectives
  - **Your contribution**: Prove approximation bounds for general nonlinear WTA
- **Parameterized complexity**: Fixed-parameter tractable algorithms when $|W|$ or $|T|$ is small
  - **Your contribution**: FPT algorithm with runtime $O(f(k) \cdot \text{poly}(n))$ for parameter $k = |W|$
- **Online algorithms**: Competitive analysis when targets arrive over time
  - **Your contribution**: Online WTA with competitive ratio analysis

Summary and Recommendations

For Your Checkpoint, Focus On:

1. **Problem Definition (Section 1)**: Present the mathematical formulation clearly with examples
2. **Complexity (Section 2)**: Sketch the SUBSET PRODUCT reduction at a high level; reference detailed proofs
3. **Algorithm Table (Section 3)**: Include the comprehensive table provided; highlight diversity of approaches
4. **Implementation Plan (Section 4)**: Select **MMR + Simulated Annealing** for implementation
5. **Experiments (Section 5)**: Design 3-4 problem sizes, 10 instances each, compare time and quality
6. **Applications (Section 6)**: Emphasize C-RAM, Iron Dome, and naval defense as compelling real-world examples
7. **Research Directions (Section 7 - Optional)**: Choose 2-3 directions that interest you (e.g., RL, TAPF, LLMs) and sketch potential contributions

Strengths of This Approach:

- Covers all checkpoint requirements comprehensively
- Balances theory (NP-hardness) with practice (algorithms, applications)
- Provides concrete implementation guidance
- Highlights cutting-edge research directions (LLMs, GNNs)

Next Steps:

1. Implement MMR and SA in Python (~200 lines total)
2. Generate random WTA instances using provided parameters

3. Run experiments and collect data
4. Create visualizations (box plots, line plots)
5. Write report integrating this material with your experimental results

This report positions your project at the intersection of classical combinatorial optimization and modern AI, demonstrating both algorithmic engineering skills and awareness of emerging research trends. The WTA problem's rich structure and practical importance make it an excellent showcase for your coursework.

⁂

1. https://portal.findresearcher.sdu.dk/files/204132463/WTA.pdf
2. https://pubsonline.informs.org/doi/10.1287/opre.1070.0440
3. https://www.semanticscholar.org/paper/11ce4409c92d58ffbc8e31a61ddfaaa9b202217b
4. https://www.semanticscholar.org/paper/a70b7fb2811cd5be9bc5b075082016c9ceb17ef4
5. https://www.foi.se/download/18.7fd35d7f166c56ebe0b10065/1542623791861/Real-time-allocation_FOI-S--3982--SE.pdf
6. https://ieeexplore.ieee.org/document/8685080/
7. https://downloads.hindawi.com/journals/complexity/2018/8623051.pdf
8. https://ieeexplore.ieee.org/document/8249748/
9. https://arxiv.org/pdf/1911.12567.pdf
10. https://www.sciencedirect.com/science/article/abs/pii/S0360835219300208
11. https://ieeexplore.ieee.org/document/11051676/
12. https://www.mdpi.com/2073-8994/12/5/824
13. https://www.intechopen.com/books/concepts-applications-and-emerging-opportunities-in-industrial-engineering/weapon-target-assignment
14. https://www.nearly42.org/cstheory/weapon-target-assignment-problem/
15. https://portal.findresearcher.sdu.dk/en/publications/weapon-target-assignment-problem-exact-and-approximate-solution-a-3
16. https://portal.findresearcher.sdu.dk/en/publications/weapon-target-assignment-problem-exact-and-approximate-solution-a-3/
17. http://www.dodccrp.org/events/10th_ICCRTS/CD/papers/182.pdf
18. https://arxiv.org/pdf/2307.00663.pdf
19. https://www.ri.cmu.edu/app/uploads/2023/08/Yimin_Tang_MSR_Thesis-2.pdf
20. https://en.wikipedia.org/wiki/Weapon_target_assignment_problem
21. https://dspace.mit.edu/handle/1721.1/7388
22. https://etd.lib.metu.edu.tr/upload/12610025/index.pdf
23. https://link.springer.com/10.1007/s10479-022-04525-6
24. https://onlinelibrary.wiley.com/doi/10.1002/nav.22249
25. https://dspace.mit.edu/bitstream/handle/1721.1/162816/Naval Research Logistics - 2025 - Bertsimas - Solving Large-Scale Weapon Target Assignment Problems in Seconds Using.pdf?sequence=2&isAllowed=y

26. http://jkimst.org/journal/view.php?doi=10.9766/KIMST.2020.23.4.381
27. https://www.hindawi.com/journals/mpe/2020/8618065/
28. https://onlinelibrary.wiley.com/doi/10.1155/2020/8618065
29. https://scholar.afit.edu/cgi/viewcontent.cgi?article=2646&context=etd
30. https://ieeexplore.ieee.org/document/5282552/
31. https://linkinghub.elsevier.com/retrieve/pii/S147466701645809X
32. https://skoge.folk.ntnu.no/prost/proceedings/ifac11-proceedings/data/html/papers/1214.pdf
33. http://thesai.org/Downloads/IJARAI/Volume2No7/Paper_7-Weapon_Target_Assignment_with_Combinatorial_Optimization_Techniques.pdf
34. https://www.matec-conferences.org/articles/matecconf/pdf/2017/42/matecconf_eitce2017_02004.pdf
35. https://onlinelibrary.wiley.com/doi/10.1155/2019/6761073
36. http://thesai.org/Downloads/Volume8No4/Paper_12-A_Parallel_Simulated_Annealing_Algorithm.pdf
37. https://thesai.org/Publications/ViewPaper?Volume=8&Issue=4&Code=IJACSA&SerialNo=12
38. http://dpi-journals.com/index.php/dtcse/article/view/17569
39. https://pris.bit.edu.cn/docs/20130514213458765789.pdf
40. http://ieeexplore.ieee.org/document/7578704/
41. https://www.hindawi.com/journals/ijae/2017/1746124/
42. https://onlinelibrary.wiley.com/doi/10.1155/2017/1746124
43. https://accscience.com/journal/IJOCTA/10/2/10.11121/ijocta.01.2020.00775
44. https://pdfs.semanticscholar.org/a50f/3fd2f5f04cfd3ea391c1bcc698ba3b654003.pdf
45. https://ieeexplore.ieee.org/document/10621019/
46. https://onlinelibrary.wiley.com/doi/10.1155/2018/8623051
47. https://www.hindawi.com/journals/ijae/2018/8302324/
48. https://www.techscience.com/cmc/v76n3/54327
49. https://file.techscience.com/files/cmc/2023/TSP_CMC-76-3/TSP_CMC_36223/TSP_CMC_36223.pdf
50. https://ieeexplore.ieee.org/document/10859049/
51. https://file.techscience.com/files/cmc/2023/TSP_CMC-76-3/TSP_CMC_41253/TSP_CMC_41253.pdf
52. https://dl.acm.org/doi/10.1145/3704558.3707086
53. https://pmc.ncbi.nlm.nih.gov/articles/PMC10839049/
54. https://arxiv.org/abs/2511.10207
55. https://arxiv.org/pdf/2511.10207.pdf
56. https://www.techrxiv.org/users/751207/articles/722348/master/file/data/main/main.pdf
57. https://ieeexplore.ieee.org/document/10662818/
58. https://www.mdpi.com/2073-4336/15/5/33
59. https://iopscience.iop.org/article/10.1088/1757-899X/569/3/032014
60. http://downloads.hindawi.com/journals/mpe/2017/9897153.pdf
61. https://ieeexplore.ieee.org/document/10681512/
62. https://open.metu.edu.tr/bitstream/handle/11511/111327/NADP_Thesis.pdf

63. https://www.reddit.com/r/explainlikeimfive/comments/glwyzz/eli5_how_do_cram_systems_work_counter_rocket/
64. https://en.wikipedia.org/wiki/C-RAM
65. https://en.wikipedia.org/wiki/Iron_Dome
66. https://link.springer.com/10.1007/s42405-024-00717-5
67. https://ieeexplore.ieee.org/document/10516683/
68. https://dl.acm.org/doi/10.1145/3580219.3580247
69. https://crl.cs.technion.ac.il/wp-content/uploads/2022/01/CoMAPF-Thesis.pdf
70. https://www.sciencedirect.com/science/article/am/pii/S030504831930711X
71. https://journals.sagepub.com/doi/pdf/10.1177/1729881420905922
72. https://ieeexplore.ieee.org/document/10429060/
73. https://patents.google.com/patent/CN115222271A/en
74. https://data.mendeley.com/datasets/jt2ppwr62p/1
75. https://www.mdpi.com/2079-9292/9/9/1511/pdf
76. http://arxiv.org/pdf/2312.12192.pdf
77. https://onlinelibrary.wiley.com/doi/10.1002/9781119266235.ch5
78. https://dl.acm.org/doi/pdf/10.1145/3284557.3284707
79. https://link.springer.com/10.1007/s42405-023-00572-w
80. https://www.hindawi.com/journals/mpe/2021/6631274/
81. http://ieeexplore.ieee.org/document/7279633/
82. http://ieeexplore.ieee.org/document/4609616/
83. https://link.springer.com/10.1007/978-3-031-38906-1_6
84. https://www.semanticscholar.org/paper/206586f251c09cc1a10405b1bc26123d76407a85
85. https://arxiv.org/pdf/0907.0067.pdf
86. https://arxiv.org/pdf/2404.08086.pdf
87. http://downloads.hindawi.com/journals/ijae/2018/8302324.pdf
88. https://scholar.afit.edu/cgi/viewcontent.cgi?article=2943&context=etd
89. https://arc.aiaa.org/doi/abs/10.2514/1.G003515
90. https://dl.acm.org/doi/abs/10.1007/s10898-020-00938-4
91. https://www.repositorio.ufop.br/items/4c2b6b54-41d6-4d05-92c5-7802f8082d98
92. https://ieeexplore.ieee.org/iel8/6287639/10820123/11175137.pdf
93. https://ieeexplore.ieee.org/document/10765848/
94. https://mcitdoc.org.ua/index.php/ITConf/article/view/635
95. https://ieeexplore.ieee.org/document/9728512/
96. http://www.dbpia.co.kr/Journal/ArticleDetail/NODE09300906
97. http://orion.journals.ac.za/pub/article/download/522/454
98. https://dc-china-simulation.researchcommons.org/journal/vol36/iss6/4/
99. https://corelab.mae.ufl.edu/papers/WTA.pdf
100. https://www.techscience.com/cmc/v76n3/54327/html

101. https://centcomcitadel.com/en_GB/articles/ssc/features/2023/10/06/feature-02
102. https://dl.acm.org/doi/fullHtml/10.1145/3580219.3580247
103. https://www.sciencedirect.com/science/article/abs/pii/S030504831930711X
104. https://www.facebook.com/WIONews/posts/israel-uses-a-6-layer-air-defence-network-including-iron-dome-davids-sling-arrow/1239885964917242/
105. https://ieeexplore.ieee.org/document/10887290/
106. https://dl.acm.org/doi/10.1145/3477495.3536327
107. https://www.mdpi.com/1424-8220/22/6/2194
108. https://ieeexplore.ieee.org/document/10976246/
109. http://vtz.asv.gov.ua/article/view/346829
110. https://deepscienceresearch.com/dsr/catalog/book/359
111. https://www.semanticscholar.org/paper/9cc7b50ea7ddfb8d24e92707a3f0cd64e175a2d7
112. https://www.mdpi.com/2076-3417/9/18/3803/pdf
113. https://www.sciencedirect.com/science/article/abs/pii/S0952197624013708
114. https://www.sciencedirect.com/science/article/pii/S2214914724002228
115. https://onlinelibrary.wiley.com/doi/full/10.1002/nav.22249
116. https://ieeexplore.ieee.org/iel7/6287639/10005208/10283838.pdf
117. image.jpg
118. image.jpg
119. https://link.springer.com/10.1007/s41870-023-01674-0
120. https://downloads.hindawi.com/journals/mpe/2020/8618065.pdf
121. https://arxiv.org/pdf/2402.08365.pdf
122. https://arxiv.org/pdf/2301.05084.pdf
123. http://arxiv.org/pdf/2203.07941.pdf
124. http://arxiv.org/pdf/2501.14705.pdf
125. https://stackoverflow.com/questions/42879631/reducing-time-complexity-of-knapsack-01-while-using-dp-algorithm
126. https://pubsonline.informs.org/doi/10.1287/ijoc.1.4.232
127. https://etd.lib.metu.edu.tr/upload/12614452/index.pdf
128. https://cp-algorithms.com/dynamic_programming/knapsack.html
129. https://dergipark.org.tr/en/pub/ctc/issue/86213/1487334
130. https://en.wikipedia.org/wiki/Knapsack_problem
131. https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/
132. https://ieeexplore.ieee.org/document/10283838/
133. https://thesai.org/Downloads/IJARAI/Volume2No7/Paper_7-Weapon_Target_Assignment_with_Combinatorial_Optimization_Techniques.pdf
134. https://www.aimspress.com/article/doi/10.3934/math.2025538?viewType=HTML
135. http://link.springer.com/10.1007/s10732-018-9401-1
136. https://www.semanticscholar.org/paper/752dc8d452b89a1055579d8974b92816005cc52e

137. https://www.semanticscholar.org/paper/18c3c8a01f05928154f9d85e6b996bd696fd4b29
138. https://onlinelibrary.wiley.com/doi/10.1002/dac.2583
139. https://journals.sagepub.com/doi/full/10.3233/JIFS-202679
140. https://arxiv.org/ftp/arxiv/papers/1401/1401.3442.pdf
141. https://jair.org/index.php/jair/article/download/10586/25329